

лабораторная работа №7

Основы информационной безопасности

Павлова Татьяна Юрьевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	10
5	Ответы на контрольные вопросы	16
6	Выводы	18

Список иллюстраций

4.1	Функция перевода тексты в числа	10
4.2	Функция для шифрования текста	11
4.3	Подбор возможных ключей для фрагмента	11
4.4	Генерация ключа	11
4.5	Основная логика	12
4.6	Результат работы программы	12

Список таблиц

1 Цель работы

Целью данной работы является освоение на практике применения режима однократного гаммирования

2 Задание

Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно: 1. Определить вид шифротекста при известном ключе и известном открытом тексте. 2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста

3 Теоретическое введение

Предложенная Г. С. Вернамом так называемая «схема однократного использования (гаммирования)» является простой, но надёжной схемой шифрования данных.
[course]

Гаммирование представляет собой наложение (снятие) на открытые (зашифрованные) данные последовательности элементов других данных, полученной с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных. Иными словами, наложение гаммы — это сложение её элементов с элементами открытого (закрытого) текста по некоторому фиксированному модулю, значение которого представляет собой известную часть алгоритма шифрования.

В соответствии с теорией криптоанализа, если в методе шифрования используется однократная вероятностная гамма (однократное гаммирование) той же длины, что и подлежащий сокрытию текст, то текст нельзя раскрыть. Даже при раскрытии части последовательности гаммы нельзя получить информацию о всём скрываемом тексте.

Наложение гаммы по сути представляет собой выполнение операции сложения по модулю 2 (XOR) (обозначаемая знаком $\oplus$) между элементами гаммы и элементами подлежащего сокрытию текста. Напомним, как работает операция XOR над битами:
 $0 \oplus 0 = 0, 0 \oplus 1 = 1, 1 \oplus 0 = 1, 1 \oplus 1 = 0$.

Такой метод шифрования является симметричным, так как двойное прибавление одной и той же величины по модулю 2 восстанавливает исходное значение, а шифрование и расшифрование выполняется одной и той же программой.

Если известны ключ и открытый текст, то задача нахождения шифротекста заключается в применении к каждому символу открытого текста следующего правила:

$$C_i = P_i \oplus K_i, (7.1)$$

где C_i — i -й символ получившегося зашифрованного послания, P_i — i -й символ открытого текста, K_i — i -й символ ключа, $i = 1, m$. Размерности открытого текста и ключа должны совпадать, и полученный шифротекст будет такой же длины.

Если известны шифротекст и открытый текст, то задача нахождения ключа решается также в соответствии с (7.1), а именно, обе части равенства необходимо сложить по модулю 2 с P_i :

$$C_i \oplus P_i = P_i \oplus K_i \oplus P_i = K_i,$$

$$K_i = C_i \oplus P_i.$$

Открытый текст имеет символьный вид, а ключ — шестнадцатеричное представление. Ключ также можно представить в символьном виде, воспользовавшись таблицей ASCII-кодов.

К. Шеннон доказал абсолютную стойкость шифра в случае, когда однократно используемый ключ, длиной, равной длине исходного сообщения, является фрагментом истинно случайной двоичной последовательности с равномерным законом распределения. Криптоалгоритм не даёт никакой информации об открытом тексте: при известном зашифрованном сообщении C все различные ключевые последовательности K возможны и равновероятны, а значит, возможны и любые сообщения P .

Необходимые и достаточные условия абсолютной стойкости шифра:

- полная случайность ключа;
- равенство длин ключа и открытого текста;
- однократное использование ключа.

Рассмотрим пример.

Ключ Центра:

05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70
54

Сообщение Центра:

Штирлиц – Вы Герой!!

D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21
21

Зашифрованный текст, находящийся у Мюллера:

DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51
75

Дешифровальщики попробовали ключ:

05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 F4 D3 07 BB BC
54

и получили текст:

D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C1 EE EB E2 E0 ED
21

Штирлиц - Вы Болван!

Другие ключи дадут лишь новые фразы, пословицы, стихотворные строфы, словом, всевозможные тексты заданной длины.

4 Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, листинг программы и результаты выполнения приведены в отчете.

Требуется разработать программу, позволяющее шифровать и дешифровать данные в режиме одноразового гаммирования. Начнем с создания функции перевода текстовых символов в числовые значения (рис. [fig:001]).

```
import random

def text_to_codes(text):
    """Преобразует строку в список кодов символов"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes
```

Рисунок 4.1: Функция перевода тексты в числа

Необходимо определить вид шифротекста при известном ключе и известном открытом тексте. Так как операция исключающего или отменяет сама себя, делаю одну функцию и для шифрования и для дешифрования текста (рис. [fig:003]).

```
def codes_to_text(codes):
    """Преобразует список кодов обратно в строку"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text
```

Рисунок 4.2: Функция для шифрования текста

Также создаю функцию преобразования чисел обратно в текст (рис. [fig:002]).

```
def xor(b1, b2):
    """XOR двух списков кодов"""
    if len(b1) != len(b2):
        print("Ошибка: длины не соответствуют")
        return None
    result = []
    for i in range(len(b1)):
        new = b1[i]^b2[i]
        result.append(new)
    return result
```

Рисунок 4.3: Подбор возможных ключей для фрагмента

Добавляю функцию генерации случайного ключа (рис. [fig:004]).

```
def generate_key(length):
    """Генерирует случайный ключ заданной длины"""
    key = []
    for i in range(length):
        key.append(random.randint(0, 255))
    return key
```

Рисунок 4.4: Генерация ключа

Проверка работы всех функций. Шифрование и дешифрование происходит верно, как и нахождение ключей, с помощью которых можно расшифровать верно только кусок текста (рис. [fig:005]). (рис. [fig:006]).

```

# основная программа
plain = ("С Новым Годом, друзья!")
print(f"Открытый текст: {plain}")

# генерация ключа
key = generate_key(len(plain))
print(f"Ключ: {key}")

# преобразование ключа в коды
plain_codes = text_to_codes(plain)

# шифрование
cipher_bytes = xor(plain_codes, key)
cipher_text = codes_to_text(cipher_bytes)
print(f"Зашифрованный текст: {cipher_text}")

# дешифровка
decrypted_bytes = xor(cipher_bytes, key)
decrypted_text = codes_to_text(decrypted_bytes)
print(f"Расшифровано: {decrypted_text}")

# поиск ключа
target = ("С Новым Годом, друзья!")
print(f"Искомый текст: {target}")
target_bytes = text_to_codes(target)

possible_key = xor(cipher_bytes, target_bytes)
print(f"Найденный ключ: {possible_key}")

```

Рисунок 4.5: Основная логика

```

[tanya@user lab07]$ python3 lab1.py
Открытый текст: С Новым Годом, друзья!
Ключ: [5, 5, 118, 18, 104, 167, 42, 47, 133, 203, 22, 125, 197, 184, 175, 40, 212, 119, 121, 98, 168, 233]
Зашифрованный текст: Ф%хьѡѢЖЧтуѢмБдюѡѢ
Расшифровано: С Новым Годом, друзья!
Искомый текст: С Новым Годом, друзья!
Найденный ключ: [5, 5, 118, 18, 104, 167, 42, 47, 133, 203, 22, 125, 197, 184, 175, 40, 212, 119, 121, 98, 168, 233]

```

Рисунок 4.6: Результат работы программы

Листинг программы:

```
import random
```

```

def text_to_codes(text):
    """XXXXXXXXXX XXXXX X XXXXXX XXXXX XXXXXXXX"""
    codes = []
    for i in range(len(text)):
        codes.append(ord(text[i]))
    return codes

def codes_to_text(codes):
    """XXXXXXXXXX XXXXXX XXXXX XXXXXXXX X XXXXXX"""
    text = ""
    for i in range(len(codes)):
        text += chr(codes[i])
    return text

def xor(b1, b2):
    """XOR XXXX XXXXXXXX XXXXX"""
    if len(b1) != len(b2):
        print("XXXXX: XXXXX X XXXXXXXXXXXXX")
        return None
    result = []
    for i in range(len(b1)):
        new = b1[i]^b2[i]
        result.append(new)
    return result

def generate_key(length):
    """XXXXXXXXXX XXXXXXXXXX XXXX XXXXXXXXXX XXXXXX"""

```

```

    key = []
    for i in range(length):
        key.append( random.randint(0, 255))
    return key

# 生成明文
plain = ("  1234567890, 1234567890!")
print(f"明文: {plain}")

# 生成密钥
key = generate_key(len(plain))
print(f"密钥: {key}")

# 将明文转换为字节
plain_codes = text_to_codes(plain)

# 加密
cipher_bytes = xor(plain_codes, key)
cipher_text = codes_to_text(cipher_bytes)
print(f"密文: {cipher_text}")

# 解密
decrypted_bytes = xor(cipher_bytes, key)
decrypted_text = codes_to_text(decrypted_bytes)
print(f"解密后的明文: {decrypted_text}")

```

```

# 解密 明文
target = (" 解密 明文, 解密!")
print(f"解密 明文: {target}")
target_bytes = text_to_codes(target)

possible_key = xor(cipher_bytes, target_bytes)
print(f"解密 明文: {possible_key}")

```

5 Ответы на контрольные вопросы

1. Поясните смысл одноразового гаммирования. - Одноразовое гаммирование - это метод шифрования, при котором каждый символ открытого текста гаммируется с соответствующим символом ключа только один раз.
2. Перечислите недостатки одноразового гаммирования. - Недостатки одноразового гаммирования:
 - Уязвимость к частотному анализу из-за сохранения частоты символов открытого текста в шифротексте.
 - Необходимость использования одноразового ключа, который должен быть длиннее самого открытого текста.
 - Нет возможности использовать один ключ для шифрования разных сообщений.
3. Перечислите преимущества одноразового гаммирования. - Преимущества одноразового гаммирования:
 - Высокая стойкость при правильном использовании случайного ключа.
 - Простота реализации алгоритма.
 - Возможность использования случайного ключа.
4. Почему длина открытого текста должна совпадать с длиной ключа? - Длина открытого текста должна совпадать с длиной ключа, чтобы каждый символ открытого текста гаммировался с соответствующим символом ключа.

5. Какая операция используется в режиме однократного гаммирования, назовите её особенности? - В режиме однократного гаммирования используется операция XOR (исключающее ИЛИ), которая объединяет двоичные значения символов открытого текста и ключа для получения шифротекста. Особенность XOR - если один из битов равен 1, то результат будет 1, иначе 0.
6. Как по открытому тексту и ключу получить шифротекст? - Для получения шифротекста по открытому тексту и ключу каждый символ открытого текста гаммируется с соответствующим символом ключа с помощью операции XOR.
7. Как по открытому тексту и шифротексту получить ключ? - По открытому тексту и шифротексту невозможно восстановить действительный ключ, так как для этого нужна информация о каждом символе ключа.
8. В чем заключаются необходимые и достаточные условия абсолютной стойкости шифра - Необходимые и достаточные условия абсолютной стойкости шифра:
- Ключи должны быть случайными и использоваться только один раз.
 - Длина ключа должна быть не менее длины самого открытого текста.
 - Ключи должны быть храниться и передаваться безопасным способом.

6 Выводы

В ходе выполнения данной лабораторной работы, мною было освоено на практике применение режима однократного гаммирования.